

ADVANCED MODEL CONTEXT PROTOCOL AND FASTMCP





MULTIPLE SERVERS

An MCP client is not limited to connecting to a single MCP server. Using the `servers` section of an `mcp.json` file, a client can configure and connect to multiple MCP servers at the same time. Each server typically provides a different set of tools or resources—for example, one server might access a file system, another might connect to GitHub, and a third might expose custom business tools.

From the user's perspective, these servers appear as a single collection of capabilities. When the client starts, it launches or connects to each server listed in the `servers` section of the `mcp.json` file. The AI can then use tools from any of the configured servers as needed, allowing applications to be built from multiple specialized MCP servers instead of one large server that does everything.

MULTIPLE CLIENTS

A major advantage of HTTP transport

Unlike stdio, where one client starts one server process, an HTTP MCP server can accept connections from many clients at the same time.

One server can support many clients.

- Each client sends independent tool requests.
- The server keeps each client's requests separate.
- Responses are returned only to the requesting client.
- Example

Key Point

Supporting multiple clients with a single server is one of the biggest advantages of HTTP transport over stdio.

MULTIPLE CLIENTS / SAME TOOL

Many clients can call the same tool simultaneously.

Client A —▶ summarize(doc1)

Client B —▶ summarize(doc2)

Client C —▶ summarize(doc3)

The server creates a separate execution for every request.

- Each execution has its own inputs.
- Results are returned to the correct client.
- Clients do not interfere with one another.

Key Point

A single tool can process many requests concurrently.

ASYNC

If one tool waits for a database or web service, the server can continue processing other requests.

Example

- Client A Database Query (20 sec) -----▶ Result
- Client B Quick Calculation —▶ Result
- Client C Read File —▶ Result

Without async, Clients B and C would wait for Client A.

Key Point

Async enables one HTTP MCP server to efficiently support many simultaneous clients.

CONCURRENCY ISSUES

Problems can occur when multiple tool executions modify the same shared resource.

Shared resources include:

- Global variables
- Files
- Databases
- Memory caches

Best Practices

- Use local variables whenever possible.
- Avoid shared global state.
- Use locks or database transactions when needed.
- Design tools to be stateless.

Key Point

MCP supports concurrent clients, but developers must still write concurrency-safe tools.

JSON PERFORMANCE

The JSON used by MCP (including JSON-RPC) is intentionally relatively flat for performance optimization. These actions only require a handful of fields, so there is little reason for deep nesting.

The exception is the data your tools return. If your MCP tool returns a complex object—for example, a GitHub repository, an email message, or a database query—the JSON inside the tool's result can be as deeply nested as needed. The MCP protocol itself remains relatively flat; only the application data may become complex.

Module completed

"Develop a passion for learning."



Excellent

You have completed the course successfully!



Congratulations